Individual level analysis to look at the relationship between ASM adherence and seizures

Table of Contents

Format data for excel spreadsheet

Load file

```
filename = 'all_patient_diaries.xlsx';

% Extract information about number of sheets in the excel
[~, sheetNames] = xlsfinfo(filename);

% Load the data from each sheet/participant as a separate variable
for i = 1:length(sheetNames)
    sheetData = xlsread(filename, sheetNames{i});
    varName = matlab.lang.makeValidName(sheetNames{i});
    assignin('base', varName, sheetData);
end

% Extract only the seizure data (i.e., column 2) and place it all into one
% variable called seizure where each column is a different participant
for i = 1:length(sheetNames)
    sheetData = xlsread(filename, sheetNames{i});
    seizure(:,i)=sheetData(:,2);
end

% As the raw data is in half days, let's shrink the temporal data such that
% it goes by days rather than half days. The below commands just takes into
% account whether there was a seizure/no seizure for a given day. For
% example, if someone had 3 seizures in a given day, under this format they
% would just be counted as having 1 seizure day for that day.
[numRows, numCols] = size(seizure);
seizure_days = zeros(numRows / 2, numCols);
    for row = 1:numRows/2
        for col = 1:numCols
            % Check if there is a positive number in the two consecutive rows
            if seizure(2*row-1, col) > 0 || seizure(2*row, col) > 0
```

```
                seizure_days(row, col) = 1;
            end
        end
    end
```

Take into account MA

Essentially within a 90 day sliding window, this takes the number of seizure days divided by 90 for an estimate of seizure risk.

```
for i = 1:191
    for j = 1:27
        MA(i,j)=sum(seizure_days(i:i+89,j));
        MA_90(i,j)=MA(i,j)/90;
    end
end

% Since we are looking at the relationship between MA and seizures on day
% sliding window + 1, the first day would be day 91 since the MA requires
% 90 days of data. Let's create a matrix of seizure days that only starts
% at day 91 for further analysis later on.
seizure_days_forecast=seizure_days(91:281,:);
```

Extract the ASM data for each subject

End of data format will be 3D matrix [Dose X ASM X Subject]

```
% Patient 4165: Column 3 Keppra, Column 4 Epidiolex, Column 5 Fluramine
ASM(:,1:3,1)=BIDMC4165(:,3:5);

% Patient 4216: Column 3 CBD oil, Column 4 Lamotrigine
ASM(:,1:2,2)=BIDMC4216(:,3:4);

% Patient 4251: Column 3 CBD oil, Column 5 Onfi
ASM(:,1,3)=BIDMC4251(:,3);
ASM(:,2,3)=BIDMC4251(:,5);

% Patient 4417: Column 5 Keppra, Column 6 Lamictal XR, Column 7 Vimpat
ASM(:,1:3,4)=BIDMC4417(:,5:7);

% Patient 4614: Column 3 Aptiom, Column 4 Xcopri
ASM(:,1:2,5)=BIDMC4614(:,3:4);

% Patient 4684: Column 5 Phenobarb, Column 6 Lamotrigine, Column 8
% Levetiracetam
ASM(:,1:2,6)=BIDMC4684(:,5:6);
ASM(:,3,6)=BIDMC4684(:,8);

% Patient 4711: Column 3 Felbatol, Column 4 Zonisamide, Column 5 Lamictal
ASM(:,1:3,7)=BIDMC4711(:,3:5);

% Patient 4722: Column 3 Felbatol, Column 4 Onfi, Column 6 Depakote, Column
% 7 Gabapentin, Column 8 Zonisamide, Column 9 Fenfluramine
```

```
ASM(:,1:2,8)=BIDMC4722(:,3:4);
ASM(:,3:6,8)=BIDMC4722(:,6:9);

% Patient 4768: Column 3 Topomax, Column 4 Epidiolex, Column 5 Lamictal,
% Column 6 Zarontin
ASM(:,1:4,9)=BIDMC4768(:,3:6);

% Patient 4835: Column 3 Oxtellar, Column 4 Epidiolex, Column 5 Clobazam,
Column 9 Fintepla
ASM(:,1:3,10)=BIDMC4835(:,3:5);
ASM(:,4,10)=BIDMC4835(:,9);

% Patient 4982: Column 3 Lamotrigine, Column 4 Banzel, Column 5 Epidiolex
ASM(:,1:3,11)=BIDMC4982(:,3:5);

% Patient 5057: Column 4 Keppra, Column 6 Onfi
ASM(:,1,12)=BIDMC5057(:,4);
ASM(:,2,12)=BIDMC5057(:,6);

% Patient 5117: Column 4 Zonisamide
ASM(:,1,13)=BIDMC5117(:,4);

% Patient 5146: Column 3 Zonisamide
ASM(:,1,14)=BIDMC5146(:,3);

% Patient 5184: Column 3 Valproic Acid, Column 4 Keppra
ASM(:,1:2,15)=BIDMC5184(:,3:4);

% Patient 5274: Column 3 Lamictal XR, Column 6 Gabapentin, Column 7
% Fycompa
ASM(:,1,16)=BIDMC5274(:,3);
ASM(:,2:3,16)=BIDMC5274(:,6:7);

% Patient 5392: Column 3 Vimpat, Column 4 Keppra, Column 5 Depakote, Column
% 6 Zonegran
ASM(:,1:4,17)=BIDMC5392(:,3:6);

% Patient 5412: Column 3 Zonegran, Column 4 Trileptal, Column 5 Fycompa
ASM(:,1:3,18)=BIDMC5412(:,3:5);

% Patient 5507: Column 3 Briviact, Column 4 Lamotrigine, Column 5
% Epidiolex, Column 6 Xcopri
ASM(:,1:4,19)=BIDMC5507(:,3:6);

% Patient 5606: Column 3 Vimpat, Column 4 Epidiolex, Column 6 Keppra,
% Column 7 Keppra XR, Column 8 XCopri
ASM(:,1:2,20)=BIDMC5606(:,3:4);
ASM(:,3:5,20)=BIDMC5606(:,6:8);

% Patient 5698: Column 3 Keppra, Column 7 Depakote ER, Column 18 XCopri
ASM(:,1,21)=BIDMC5698(:,3);
ASM(:,2,21)=BIDMC5698(:,7);
ASM(:,3,21)=BIDMC5698(:,18);
```

```
% Patient 5708: Column 9 Keppra, Column 14 XCopri, Column 20 Lamotrigine,
% Column 21 Clobazam, Column 26 Epidiolex
ASM(:,1,22)=BIDMC5708(:,9);
ASM(:,2,22)=BIDMC5708(:,14);
ASM(:,3:4,22)=BIDMC5708(:,20:21);
ASM(:,5,22)=BIDMC5708(:,26);

% Patient 5731: Column 3 Carbamazepine, Column 4 Levetiracetam, Column 5
% Vimpat, Column 6 repeat Vimpat?? will ignore for now given no negative
% values/missed doses anyways
ASM(:,1:3,23)=BIDMC5731(:,3:5);

% Patient 5782: Column 3 Vimpat, Column 6 XCopri
ASM(:,1,24)=BIDMC5782(:,3);
ASM(:,2,24)=BIDMC5782(:,6);

% Patient 5790: Column 3 Keppra
ASM(:,1,25)=BIDMC5790(:,3);

% Patient 5799: Column 3 Keppra, Column 5 Tegretol XR
ASM(:,1,26)=BIDMC5799(:,3);
ASM(:,2,26)=BIDMC5799(:,5);

% Patient 59589: Column 3 Topamax, Column 4 Briviact, Column 5 Vimpat
ASM(:,1:3,27)=BIDMC59589(:,3:5);
```

Matrix of missed doses

The raw data for the ASM data is in half days. Let's similarly shrink the temporal resolution such that we look at ASMs in days rather than half days. We will account for missed doses in terms of "administration periods". I.e., if an individual missed morning doses of three different ASMs, that only counts as one missed administration period. For the possible range of values here, for each day it can be 0 (no missed doses), 1 (1 missed administration period, i.e., morning vs night), and 2 (missed both morning and night doses if scheduled for it).

```
[numRows, numCols, numSubs] = size(ASM);
Missed_Doses = zeros(numRows / 2, numCols, numSubs);
    for row = 1:numRows/2
        for col = 1:numCols
            for subs = 1:numSubs
            if ASM(2*row-1, col, subs) < 0 && ASM(2*row, col, subs) < 0
               Missed_Doses(row, col, subs) = 2;
            elseif ASM(2*row-1, col, subs) < 0 && ASM(2*row, col, subs) >= 0
|| ASM(2*row-1, col, subs) >= 0 && ASM(2*row, col, subs) < 0
               Missed_Doses(row, col, subs) = 1;
            end
        end
        end
    end

for j = 1:27
     Storage=Missed_Doses(:,:,j);
     Missed_Doses_Periods(:,j)=max(Storage,[],2);
end
```

```
% Since the first 90 days are used to estimate MA, for the missed doses,
% the first number used should be on day 90 to look at how it influences
% seizure risk on day 91. The last day used should be day 280, since day
% 281 is the last day to look at seizure risk.
Missed_Doses_Periods_clean=Missed_Doses_Periods(90:280,:);

% Let's allow the creation of lag terms to look at how the effect of missed
% doses in the past influence seizure occurrence.
max_lag = 6; %in days, 6 because the data is already lagged by 1 day per
above
n_subjects = 27;

% The maximum number of predictors is max_lag + 1 (for the raw variable) + 1
(for the intercept).
B_optimal_allsubs = zeros(max_lag + 2, n_subjects);
p_allsubs = zeros(max_lag + 2, n_subjects);
ci_lower_optimal_allsubs = zeros(max_lag + 2, n_subjects);
ci_upper_optimal_allsubs = zeros(max_lag + 2, n_subjects);
chi2_p_allsubs = zeros(1, n_subjects); % Stores the chi-square p-value for
the optimal model
optimal_lags_allsubs = cell(1, n_subjects);

for i = 1:n_subjects
    % Select subject-specific data
    ASM_var = Missed_Doses_Periods_clean(:, i);
    Seizure_var = seizure_days_forecast(:, i);
    MA_var = MA_90(:, i);

    % Set X as MA or ASM, and Y as Seizure_var
    X = ASM_var;
    Y = Seizure_var;

    % --- FEATURE SCALING (Z-TRANSFORM) ---
    % Scale the predictor variable X using a z-transform (mean=0, std=1)
    mu_X = mean(X, 'omitnan');
    sigma_X = std(X, 'omitnan');

    % Check for zero variance, which indicates no missed doses for this
subject
    if sigma_X < 1e-10
        fprintf('□ Subject %d: No variability in missed doses. Skipping
model fitting.\n\n', i);
        B_optimal_allsubs(:, i) = NaN;
        p_allsubs(:, i) = NaN;
        ci_lower_optimal_allsubs(:, i) = NaN;
        ci_upper_optimal_allsubs(:, i) = NaN;
        chi2_p_allsubs(i) = NaN;
        continue; % Skip to the next subject
    end

    X_scaled = (X - mu_X) / sigma_X;

    %FIND OPTIMAL LAG COMBINATION USING AIC
```

```
    min_AIC = Inf;
    best_combo_AIC = [];

    fprintf('--- Subject %d: Finding Optimal Lag Combination ---\n', i);

    % Generate all possible lag combinations from 1 to max_lag
    lag_indices = 1:max_lag;
    valid_models_found = false;

    % Loop through all possible combinations of lags
    for k = 1:max_lag
        lag_combinations = nchoosek(lag_indices, k);
        for combo_idx = 1:size(lag_combinations, 1)
            current_lags = lag_combinations(combo_idx, :);

            % The max lag in the current combination determines the data
trimming.
            current_max_lag = max([0, current_lags]);

            % Build the lagged predictor matrix for the current combination
            X_valid = buildLaggedMatrix(X_scaled, current_lags,
current_max_lag);

            % Trim the response variable
            y_valid = Y(current_max_lag+1:end);

            % Ensure enough observations to fit the model
            if size(X_valid, 1) <= (k + 1)
                fprintf('Skipped combination {%s}: Not enough
observations.\n', num2str(current_lags));
                continue;
            end

            try
                % Fit logistic regression model
                [B, dev, stats] = glmfit(X_valid, y_valid, 'binomial',
'link', 'logit', 'LikelihoodPenalty', 'jeffreys-prior');

                % Skip if unstable
                if any(isnan(B))
                    fprintf('Skipped combination {%s}: NaN coefficients.\n',
num2str(current_lags));
                    continue;
                end

                valid_models_found = true;

                % Calculate AIC
                num_params = length(B);
                current_AIC = 2 * num_params + dev;

                % Update optimal model based on AIC
                if current_AIC < min_AIC
                    min_AIC = current_AIC;
```

```
                    best_combo_AIC = current_lags;
                end

            catch ME
                fprintf('Skipped combination {%s} due to error: %s\n',
num2str(current_lags), ME.message);
                continue;
            end
        end
    end

    %DETERMINE AND FIT THE OPTIMAL MODEL
    if valid_models_found
        % Use the AIC-selected model.
        optimal_lags = best_combo_AIC;
        optimal_lags_allsubs{i} = optimal_lags;

        fprintf('\n □ Optimal lag combination: {%s} (AIC = %.2f)\n',
num2str(optimal_lags), min_AIC);

        % Build the final predictor matrix for the optimal model
        optimal_max_lag = max([0, optimal_lags]);
        X_opt = buildLaggedMatrix(X_scaled, optimal_lags, optimal_max_lag);
        y_opt = Y(optimal_max_lag+1:end);

        % Fit the final logistic regression model
        [B_optimal, dev_optimal, stats_optimal] = glmfit(X_opt, y_opt,
'binomial', 'link', 'logit', 'LikelihoodPenalty', 'jeffreys-prior');

        %MODEL SIGNIFICANCE TEST (CHI-SQUARE)
        [~, dev_null, ~] = glmfit(ones(size(y_opt)), y_opt, 'binomial',
'link', 'logit', 'LikelihoodPenalty', 'jeffreys-prior');
        chi2_statistic = dev_null - dev_optimal;
        df = size(X_opt, 2);
        chi2_p_value = 1 - chi2cdf(chi2_statistic, df);

        %CALCULATE CONFIDENCE INTERVALS
        se_optimal = stats_optimal.se;
        df_optimal = stats_optimal.dfe;
        alpha = 0.05;
        t_critical = tinv(1 - alpha / 2, df_optimal);
        ci_lower_optimal = B_optimal - t_critical * se_optimal;
        ci_upper_optimal = B_optimal + t_critical * se_optimal;

        % Store the results
        B_optimal_allsubs(1:length(B_optimal), i) = B_optimal;
        p_allsubs(1:length(stats_optimal.p), i) = stats_optimal.p;
        ci_lower_optimal_allsubs(1:length(ci_lower_optimal), i) =
ci_lower_optimal;
        ci_upper_optimal_allsubs(1:length(ci_upper_optimal), i) =
ci_upper_optimal;
        chi2_p_allsubs(i) = chi2_p_value;

        % Display the results
```

```
        fprintf('\nFinal Model Results for Subject %d\n', i);
        fprintf('Optimal Lags: {%s}\n', num2str(optimal_lags));
        disp('Final Model Coefficients:');
        disp(B_optimal');
        disp('Final Model P-values:');
        disp(stats_optimal.p');
        fprintf('Chi-square Model p-value: %.4f\n\n', chi2_p_value);
    else
        fprintf('□ No valid models could be fitted for subject %d.\n', i);
        B_optimal_allsubs(:, i) = NaN;
        p_allsubs(:, i) = NaN;
        ci_lower_optimal_allsubs(:, i) = NaN;
        ci_upper_optimal_allsubs(:, i) = NaN;
        chi2_p_allsubs(i) = NaN;
    end
end

%HELPER FUNCTION: BUILD LAGGED MATRIX
function lagged_matrix = buildLaggedMatrix(X, lags, max_lag)
    % Builds a predictor matrix with the unlagged variable and selected lags.
    % All variables are trimmed to the same length.

    % Initialize matrix with the unlagged, trimmed variable
    lagged_matrix = X(max_lag+1:end);

    % Add lagged variables, each trimmed to the same length as the first.
    for l = lags
        % The start index for a lagged variable is adjusted by the maximum
lag.
        start_idx = max_lag - l + 1;
        end_idx = length(X) - l;
        lagged_var = X(start_idx:end_idx);
        lagged_matrix = [lagged_matrix, lagged_var];
    end
end
```

Spaghetti Plot of Seizure Occurrence

Check if the necessary variables exist

```
if ~exist('Missed_Doses_Periods_clean', 'var') ||
~exist('seizure_days_forecast', 'var')
    error('Required variables Missed_Doses_Periods_clean or
seizure_days_forecast not found. Please run the preceding code first.');
end
num_subjects = size(Missed_Doses_Periods_clean, 2);
num_days_to_plot = min(size(Missed_Doses_Periods_clean, 1),
size(seizure_days_forecast, 1));

% Initialize matrices to store percentages and standard errors for each
subject
seizure_percentages_all = zeros(num_subjects, 2);
se_all = zeros(num_subjects, 2);
```

```
% Loop through each subject
for i = 1:num_subjects
    % Initialize variables for the current subject
    seizure_counts_subj = zeros(1, 2);
    total_counts_subj = zeros(1, 2);
    % Loop through each day for the current subject
    for j = 1:num_days_to_plot
        missed_dose_period = Missed_Doses_Periods_clean(j, i);
        seizure_on_lagged_day = seizure_days_forecast(j, i);
        % Categorize into two groups: 0 or >0 missed doses
        if missed_dose_period == 0
            index = 1; % No missed doses
        else
            index = 2; % One or two missed doses
        end
        % Add to total count for this category for the current subject
        total_counts_subj(index) = total_counts_subj(index) + 1;
        % If there was a seizure on the lagged day, add to the seizure count
        if seizure_on_lagged_day == 1
            seizure_counts_subj(index) = seizure_counts_subj(index) + 1;
        end
    end
    % Calculate the percentage and standard error for the current subject
    seizure_percentages_all(i, :) = (seizure_counts_subj ./
total_counts_subj) * 100;
    se_all(i, :) = sqrt(seizure_percentages_all(i, :) .* (100 -
seizure_percentages_all(i, :)) ./ total_counts_subj);
end

% Create a new figure
figure;
set(gcf, 'Color', 'w'); % Sets the figure background to white
set(gca, 'Color', 'w'); % Sets the axes background to white
hold on;

% Initialize handles for the legend
h_individual = [];
h_subj23 = [];

% Plot each subject's data
for i = 1:num_subjects
    % Check for NaN values and skip subjects with no data for missed doses
    if ~isnan(seizure_percentages_all(i, 2))
        if i == 23
            % Plot subject 23: light grey mixed with red, dashed line
            h_line = plot(1:2, seizure_percentages_all(i, :), '--', 'Color',
[0.85 0.6 0.6], 'LineWidth', 1);
            h_subj23 = h_line; % Store the handle for the legend
        else
            % Plot all other subjects: dashed line in gray
            h_line = plot(1:2, seizure_percentages_all(i, :), '--', 'Color',
[0.7 0.7 0.7], 'LineWidth', 1);
            if isempty(h_individual)
```

```
                h_individual = h_line; % Store the handle of the first gray
line
            end
        end
    end
end

% Calculate the group-level averages for plotting
group_avg = nanmean(seizure_percentages_all);
group_se = nanstd(seizure_percentages_all) ./
sqrt(sum(~isnan(seizure_percentages_all)));

% Plot the group-level average as a bold line and get its handle
h_group = plot(1:2, group_avg, 'k', 'LineWidth', 3);

% Add error bars for the group average
errorbar(1:2, group_avg, group_se, 'k', 'LineStyle', 'none', 'LineWidth', 2);
hold off;

% Customize the plot
xlim([0.5 2.5]);
xticks(1:2);
xticklabels({'No Missed Doses', 'Missed Doses'});
title('Individual and Group-Level Seizure Occurrence');
ylabel('Seizure Occurrence (%)');

% Correct the legend using the handles
legend([h_individual, h_group, h_subj23], {'Other patients', 'Group
Average', 'Patient with ASM-related seizures'}, 'Location', 'best');
grid on;
set(gca, 'FontSize', 12);
%print(gcf, 'Spaghetti_Plot_Seizure_Occurrence_Modified.tif', '-dtiff', '-
r300');
```

Published with MATLAB® R2024b